# Boxplots: from distributions to models

A Course on Data Visualization

Livio Finos, University of Padova

## Contents

**Learning goals and lesson route** **2**

**Reading a boxplot, element by element** **3**

Start with the observations . . . 3

What a boxplot does not tell us . . . 4

**Skewed data: how many "outliers" should we expect?** **6**

One rule, two generating models . . . 6

A population benchmark . . . 7

Repeated samples: count the flags, not just one realization . . . 9

How does sample size affect the points flagged by a boxplot? . . . 12

The percentage flagged across repeated samples . . . 12

More flagged observations does not imply a higher percentage . . . 14

How often is at least one observation flagged? . . . 16

How to interpret the simulation . . . 18

**Whenever feasible, show the points as well** **19**

Boxplot plus observations . . . 19

Same boxplot, different internal structure . . . 20

The same example in ggplot2 . . . 20

**Take-home messages** **22**

**Sources and further reading** **23**

```
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = TRUE,
                      fig.width = 8, fig.height = 5, fig.align = "center",
                      dpi = 120, comment = "#>")
options(width = 90)
# Examples use both base R graphics and ggplot2.
# knitr and rmarkdown are also needed to knit the document.
```

```r
library(ggplot2)
blue <- "#0072B2"
orange <- "#D55E00"
green <- "#009E73"
purple <- "#CC79A7"

# Use exactly the same hinges and fences as base R's boxplot().
# Restrict this teaching helper to finite numeric data.
bp_info <- function(z) {
  stopifnot(is.numeric(z), length(z) >= 2L, all(is.finite(z)))
  s <- boxplot.stats(z, coef = 1.5)$stats
  spread <- s[4] - s[2]
  lower <- s[2] - 1.5 * spread
  upper <- s[4] + 1.5 * spread
  list(stats = s, iqr = spread, lower = lower, upper = upper,
       flag = z < lower | z > upper)
}

# Overlay all observations, with jitter only along the categorical axis.
# The explicit ylim preserves the full range when outline = FALSE.
box_points <- function(z, main = "", ylab = "Value", col = blue) {
  boxplot(z, outline = FALSE, ylim = range(z), xlim = c(0.6, 1.4),
          xaxt = "n", ylab = ylab, main = main, col = "grey95",
          border = "grey20", boxwex = 0.3, lwd=2)
  points(jitter(rep(1, length(z)), amount = 0.08), z,
         pch = 16, cex = 0.65, col = adjustcolor(col, alpha.f = 0.5))
}
```

## Learning goals and lesson route

A boxplot is a compact summary of a distribution. Its isolated points invite a question, not an automatic decision to remove observations.

By the end of this unit, you should be able to:

- identify the median, quartiles, interquartile range, fences and whiskers;
- explain why the proportion of boxplot-flagged observations depends on the generating distribution and why the number also depends on sample size;
- distinguish counts, percentages and the probability of at least one flag;
- distinguish variation between samples from Monte Carlo uncertainty;
- combine a boxplot with individual observations and recognise what the summary can conceal.

**Prerequisites:** quantiles, basic probability distributions and basic R.

**Unit route:** read the elements of a boxplot, compare uncontaminated samples from two distributions, investigate the effects of sample size and simulation repetitions, and use individual observations to reveal what a boxplot hides.

**Reproducibility.** Every random example has a seed. No external dataset or network connection is needed when knitting. Install `rmarkdown`, `knitr` and `ggplot2` once if necessary, then use RStudio's **Knit** button or:

```r
install.packages(c("rmarkdown", "knitr", "ggplot2"))  # run once, if needed
rmarkdown::render("05_boxplots_from_distributions_to_models.Rmd")
```

Rendering also requires Pandoc, which is bundled with RStudio. The unit combines base R graphics with ggplot2; all datasets are generated in the code.

# Reading a boxplot, element by element

## Start with the observations

Consider the following small dataset. Before plotting, predict the median and identify the largest observation.

```r
z <- c(2, 4, 5, 6, 7, 8, 9, 10, 12, 14, 24)
z
```

```
#>  [1]  2  4  5  6  7  8  9 10 12 14 24
```

```r
quantile(z, c(0.25, 0.50, 0.75))
```

```
#>  25%  50%  75%
#>  5.5  8.0 11.0
```

Let $Q_1$ and $Q_3$ denote the lower and upper quartiles, and let

$$\mathrm{IQR} = Q_3 - Q_1.$$

The **interquartile range** measures the spread of the middle part of the data. For the usual $1.5\,\mathrm{IQR}$ rule, the lower and upper **fences** are

$$L = Q_1 - 1.5\,\mathrm{IQR}, \qquad U = Q_3 + 1.5\,\mathrm{IQR}\,.$$

| Element | How to read it |
|---|---|
| Lower and upper edges of the box | Lower and upper quartiles; the box spans roughly the middle 50% of observations. |
| Line inside the box | The median, not generally the mean. |
| Length of the box along the measurement axis | The interquartile range. |
| Lower whisker | The smallest observed value greater than or equal to the lower fence. |
| Upper whisker | The largest observed value less than or equal to the upper fence. |
| Isolated points | Observations strictly outside the fences. |

**Fences are thresholds; whiskers end at observations.** A whisker therefore usually stops *before* its fence. It does not necessarily reach the sample minimum or maximum. An observation exactly on a fence is not flagged.

**R convention.** Base R uses Tukey's *hinges* as the box edges. They are quartile-like summaries and can differ from `quantile()`'s default quartiles for even sample sizes. In this small example they agree. Throughout the

base R examples and simulations, `boxplot()` and `bp_info()` use `boxplot.stats()`, so the displayed boxes and the flags use the same hinges. The ggplot2 displays use its default sample-quartile convention, which can differ slightly from base R's hinges. The ggplot2 summaries of simulation percentages do not change how the original observations were flagged. In the formulas below, $Q_1, Q_3$ refer to these sample box edges or, where explicitly stated, to population quartiles. See the R documentation for boxplot statistics.

```
bz <- bp_info(z)
anatomy <- data.frame(
  Element = c("Lower fence", "Lower whisker", "Lower box edge",
              "Median", "Upper box edge", "Upper whisker", "Upper fence"),
  Value = c(bz$lower, bz$stats, bz$upper)
)
knitr::kable(anatomy, digits = 2)
```

| Element | Value |
|---|---|
| Lower fence | -2.75 |
| Lower whisker | 2.00 |
| Lower box edge | 5.50 |
| Median | 8.00 |
| Upper box edge | 11.00 |
| Upper whisker | 14.00 |
| Upper fence | 19.25 |

```
z[bz$flag]
```

```
#> [1] 24
```

```
local({
  par(mar = c(2, 4, 3, 1))
  boxplot(z, at = 1, boxwex = 0.35, xlim = c(0.5, 4.3),
          ylim = c(-5, 27), xaxt = "n", ylab = "Observed value",
          col = "grey90", border = "grey25", main = "A boxplot, unpacked")
  segments(0.65, c(bz$lower, bz$upper), 1.5, c(bz$lower, bz$upper),
           lty = 2, col = orange)
  y_pos <- c(bz$lower, bz$stats, bz$upper, 24)
  label <- c("Lower fence = -2.75", "Lower whisker = 2",
             "Lower box edge = 5.5", "Median = 8",
             "Upper box edge = 11", "Upper whisker = 14",
             "Upper fence = 19.25", "Flagged observation = 24")
  arrows(1.65, y_pos, 1.22, y_pos, length = 0.06, col = "grey45")
  text(1.75, y_pos, labels = label, adj = 0, cex = 0.9)
})
```

Here the IQR is 5.5, the upper fence is 19.25, and the upper whisker ends at 14. The value 24 lies beyond the fence and is drawn separately.

## What a boxplot does not tell us

A standard boxplot does not show the sample size, individual data locations, clusters or gaps. Two rather different datasets can share the same boxplot. The median is not a measure of spread, the whiskers are not standard errors, and the box is not a confidence interval.

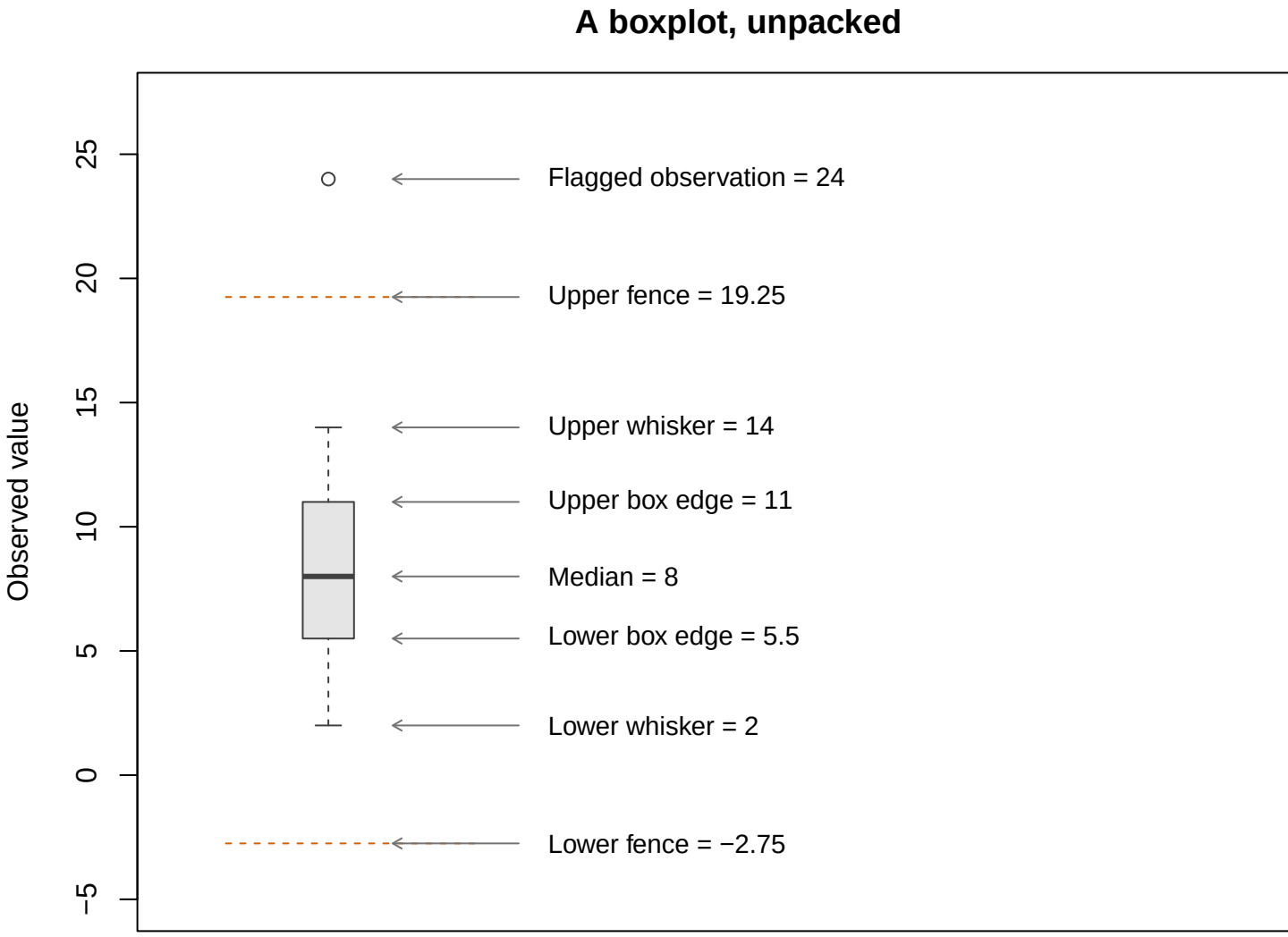



Figure 1: Anatomy of the boxplot. Dashed fences are added for teaching; they are normally invisible.

Asymmetry in the box or whiskers can suggest skewness, but a small boxplot is not a complete description of distributional shape. A histogram, density plot, ECDF or point layer can supply missing information.

> **Pause and discuss.** Is 24 a recording error? Could it be a valid observation? Can the boxplot alone answer these questions?

**Discussion.** The graph only establishes that 24 is far from the central part of this sample according to one descriptive rule. We need subject-matter information and a model to decide what that means.

# Skewed data: how many "outliers" should we expect?

## One rule, two generating models

We will compare independent observations generated from:

$$Y \sim N(0,1), \qquad Y \sim \chi^2_2.$$

The normal distribution is symmetric. The chi-squared distribution with two degrees of freedom is non-negative and strongly right-skewed; equivalently, it is exponential with mean 2. We deliberately add **no contamination** to either model.

The distributions differ in location and scale as well as shape. However, shifting a dataset or multiplying it by a positive constant does not change which observations the IQR rule flags: the box edges, fences and observations all undergo the same transformation. The difference in flagging behavior is therefore not explained merely by a change of measurement units.

We will call observations outside the fences **boxplot-flagged observations**. The shorter word *outlier* can obscure whether we mean a graphical flag, an unusual event under a model, or a data error.

```
set.seed(20260917)
n <- 1000
samples <- list(Normal = rnorm(n), `Chi-squared (df = 2)` = rchisq(n, df = 2))

one_sample <- do.call(rbind, lapply(names(samples), function(nm) {
  b <- bp_info(samples[[nm]])
  data.frame(Model = nm, n = length(samples[[nm]]),
             Lower_fence = b$lower, Upper_fence = b$upper,
             Flagged = sum(b$flag), Percent = 100 * mean(b$flag))
}))
knitr::kable(one_sample, digits = 3, row.names = FALSE)
```

| Model | n | Lower_fence | Upper_fence | Flagged | Percent |
|---|---|---|---|---|---|
| Normal | 1000 | -2.630 | 2.646 | 8 | 0.8 |
| Chi-squared (df = 2) | 1000 | -2.722 | 6.172 | 54 | 5.4 |

```
local({
  par(mfrow = c(1, 2), mar = c(3, 4, 3, 1))
  for (j in seq_along(samples)) {
    v <- samples[[j]]
    b <- bp_info(v)
```

```
    boxplot(v, col = c("#B9DDEE", "#F1CAB8")[j],
            main = names(samples)[j], ylab = "Y", xaxt = "n",
            ylim = range(c(v, b$lower, b$upper)))
    abline(h = c(b$lower, b$upper), col = orange, lty = 2)
    mtext(paste(sum(b$flag), "flagged out of", length(v)), side = 1, line = 1)
  }
})
```

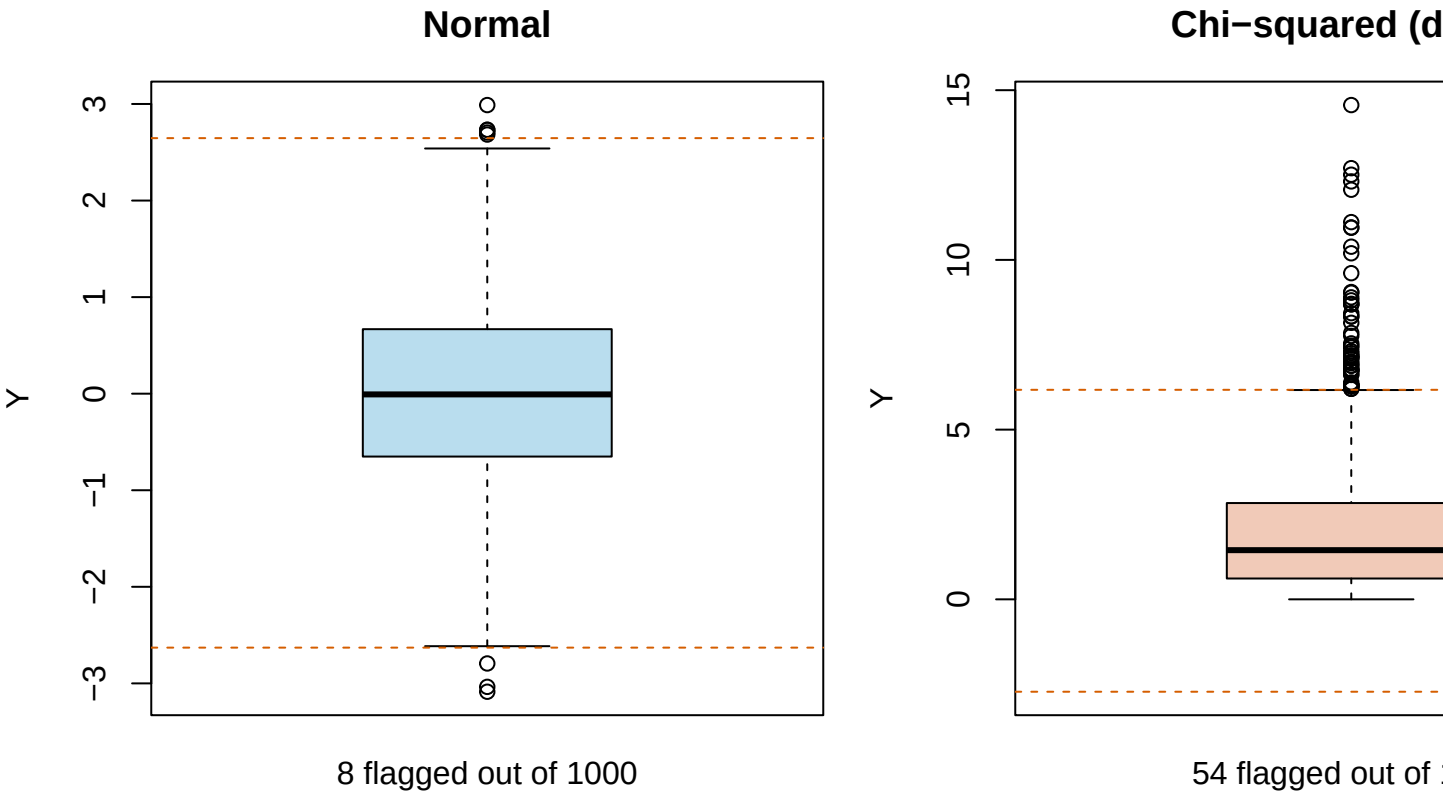



Figure 2: The same boxplot rule applied to two uncontaminated samples. The vertical scales differ; compare the pattern of flags, not absolute box heights.

**Prediction before simulation:** which model should produce more flags, and on which side? Would your prediction change if the sample size increased?

## A population benchmark

First imagine that we know the population quartiles. For a continuous population CDF $F$, define fixed population fences $L_F, U_F$ and

$$p_F = P(Y < L_F) + P(Y > U_F) = F(L_F) + 1 - F(U_F).$$

This is a **model-dependent probability**. The multiplier 1.5 does not imply a universal tail probability or a fixed significance level.

Let's compute the theoretical probability.

$$L = Q_1 - 1.5\,IQR, \qquad U = Q_3 + 1.5\,IQR.$$

since, $Q_1 = -0.67449$, $Q_3 = 0.67449$ and $IQR = 2\Phi^{-1}(0.75)\,\sigma \approx 1.34898\,\sigma$:

| Inferior Fence $L$ | Superior Fence $U$ | $P(Y < L)$ | $P(Y > U)$ | $P((Y < L) or (Y > U))$ | |
|---|---|---|---|---|---|
| $N(0,1)$ | -2,69796 | 2,69796 | 0,34883% | 0,34883% | **0,69766%** |

$$P(Y < L \text{ o } Y > U) = 2\{1 - \Phi(2.69796)\} \simeq 0.0069766.$$

```
population_rule <- function(label, qfun, pfun) {
  q <- qfun(c(0.25, 0.75))
  spread <- diff(q)
  lower <- q[1] - 1.5 * spread
  upper <- q[2] + 1.5 * spread
  p <- pfun(lower) + pfun(upper, lower.tail = FALSE)
  data.frame(Model = label, Q1 = q[1], Q3 = q[2],
             Lower_fence = lower, Upper_fence = upper, p_flag = p)
}
bench <- rbind(
  population_rule("Normal", qnorm, pnorm),
  population_rule("Chi-squared (df = 2)",
                  function(p) qchisq(p, df = 2),
                  function(q, lower.tail = TRUE)
                    pchisq(q, df = 2, lower.tail = lower.tail))
)
bench$Expected_count_n1000 <- 1000 * bench$p_flag
bench$Pr_any_n1000 <- 1 - (1 - bench$p_flag)^1000
knitr::kable(bench, digits = 4, row.names = FALSE)
```

| Model | Q1 | Q3 | Lower_fence | Upper_fence | p_flag | Expected_count_n1000 | Pr_any_n1000 |
|---|---|---|---|---|---|---|---|
| Normal | -0.6745 | 0.6745 | -2.6980 | 2.6980 | 0.0070 | 6.9766 | 0.9991 |
| Chi-squared (df = 2) | 0.5754 | 2.7726 | -2.7205 | 6.0684 | 0.0481 | 48.1125 | 1.0000 |

For a standard normal population, the fences are approximately $\pm 2.698$, and about **0.70%** of observations fall outside them. This corresponds to about **7 observations per 1000**.

For a $\chi^2_2$ population, the upper fence is approximately 6.068 and the lower fence is negative. Since the distribution cannot generate negative values, all flags are in the upper tail. About **4.81%** of observations exceed the upper fence: about **48 observations per 1000**.

For this particular chi-squared distribution, the calculation can also be done by hand. Since $F(y) = 1 - e^{-y/2}$ for $y \geq 0$,

$$Q_1 = 2\log(4/3), \quad Q_3 = 2\log 4, \quad \text{IQR} = 2\log 3,$$

and

$$U_F = 2\log 4 + 3\log 3, \qquad P(Y > U_F) = \frac{1}{4\,3^{3/2}} \simeq 0.0481.$$

With **fixed population fences** and independent observations, the flagged count is binomial with parameters $n, p_F$. Its expectation is $np_F$, and the probability of at least one flag is $1-(1-p_F)^n$. For a normal sample of size 1000, this latter probability is approximately **99.91%**. For comparison, with $n = 200$ it is approximately **75.35%**. Even under normality, *at least one* flag becomes very likely as the sample grows. These calculations use fixed population fences.

**Important distinction:** an actual boxplot estimates its fences from the same sample it classifies. Its flags are dependent, its count is not generally binomial, and the expressions above are population benchmarks rather than exact finite-sample answers. We use simulation to evaluate actual boxplots.

## Repeated samples: count the flags, not just one realization

For each model and sample size, repeat the following experiment:

1. Draw a new uncontaminated sample.
2. Compute that sample's boxplot fences.
3. Count observations below and above the fences.
4. Repeat and summarize the distribution of counts.

```
set.seed(314159)
B <- params$simulation_reps
# Validate before conversion, so a fractional value is not silently truncated.
stopifnot(is.numeric(B), length(B) == 1L, is.finite(B),
          B >= 100, B <= .Machine$integer.max, B == floor(B))
B <- as.integer(B)
sizes <- c(50L, 200L, 1000L)
generators <- list(Normal = function(n) rnorm(n),
                   `Chi-squared (df = 2)` = function(n) rchisq(n, df = 2))

sim_parts <- list()
k <- 0L
for (model in names(generators)) {
  for (nn in sizes) {
    counts <- replicate(B, {
      v <- generators[[model]](nn)
      b <- bp_info(v)
      c(lower = sum(v < b$lower), upper = sum(v > b$upper))
    })
    k <- k + 1L
    sim_parts[[k]] <- data.frame(
      Model = model, n = nn, repetition = seq_len(B),
      lower = counts["lower", ], upper = counts["upper", ]
    )
  }
}
sim <- do.call(rbind, sim_parts)
sim$count <- sim$lower + sim$upper
sim$proportion <- sim$count / sim$n

sim_summary <- do.call(rbind, lapply(
  split(sim, interaction(sim$Model, sim$n, drop = TRUE)), function(d) {
    data.frame(Model = d$Model[1], n = d$n[1],
               Mean_count = mean(d$count),
```

```
                Mean_percent = 100 * mean(d$proportion),
                SD_percent = 100 * sd(d$proportion),
                MCSE_percent = 100 * sd(d$proportion) / sqrt(nrow(d)),
                Pr_any = mean(d$count > 0),
                Count_q05 = unname(quantile(d$count, 0.05)),
                Count_q95 = unname(quantile(d$count, 0.95)))
  }
))
sim_summary <- sim_summary[order(sim_summary$n, sim_summary$Model), ]
knitr::kable(sim_summary, digits = 3, row.names = FALSE)
```

| Model | n | Mean_count | Mean_percent | SD_percent | MCSE_percent | Pr_any | Count_q05 | Count_q95 |
|---|---|---|---|---|---|---|---|---|
| Chi-squared (df = 2) | 50 | 2.434 | 4.868 | 3.144 | 0.070 | 0.906 | 0.00 | 5 |
| Normal | 50 | 0.595 | 1.190 | 2.054 | 0.046 | 0.363 | 0.00 | 3 |
| Chi-squared (df = 2) | 200 | 9.637 | 4.819 | 1.575 | 0.035 | 1.000 | 5.00 | 15 |
| Normal | 200 | 1.582 | 0.791 | 0.766 | 0.017 | 0.723 | 0.00 | 5 |
| Chi-squared (df = 2) | 1000 | 48.492 | 4.849 | 0.718 | 0.016 | 1.000 | 37.00 | 60 |
| Normal | 1000 | 7.221 | 0.722 | 0.333 | 0.007 | 0.997 | 2.95 | 13 |

`Mean_percent` is the average percentage flagged across samples. `SD_percent` describes how much that percentage varies *between samples*. `MCSE_percent` is `SD_percent / sqrt(B)` and measures numerical uncertainty in the estimated average. Both measures are expressed in percentage points. The 5th and 95th percentiles describe variability of the *counts between samples*; they are not confidence limits for the average.

Increasing $n$ changes the sampling distribution of these results. Increasing $B$ estimates that distribution more precisely without changing it.

```
local({
  par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
  focus <- subset(sim, n == 1000)
  xmax <- max(focus$count)
  probs <- lapply(names(generators), function(nm) {
    d <- focus$count[focus$Model == nm]
    tabulate(d + 1L, nbins = xmax + 1L) / length(d)
  })
  ymax <- max(unlist(probs)) * 1.1
  for (j in seq_along(probs)) {
    plot(0:xmax, probs[[j]], type = "h", lwd = 4,
         xlim = c(0, xmax), ylim = c(0, ymax),
         col = c(blue, orange)[j], main = names(generators)[j],
         xlab = "Number flagged in one sample", ylab = "Relative frequency")
    p_reference <- bench$p_flag[match(names(generators)[j], bench$Model)]
    abline(v = 1000 * p_reference, lty = 2)
    legend("topright", "Population-fence mean", lty = 2, bty = "n", cex = 0.8)
  }
})
```

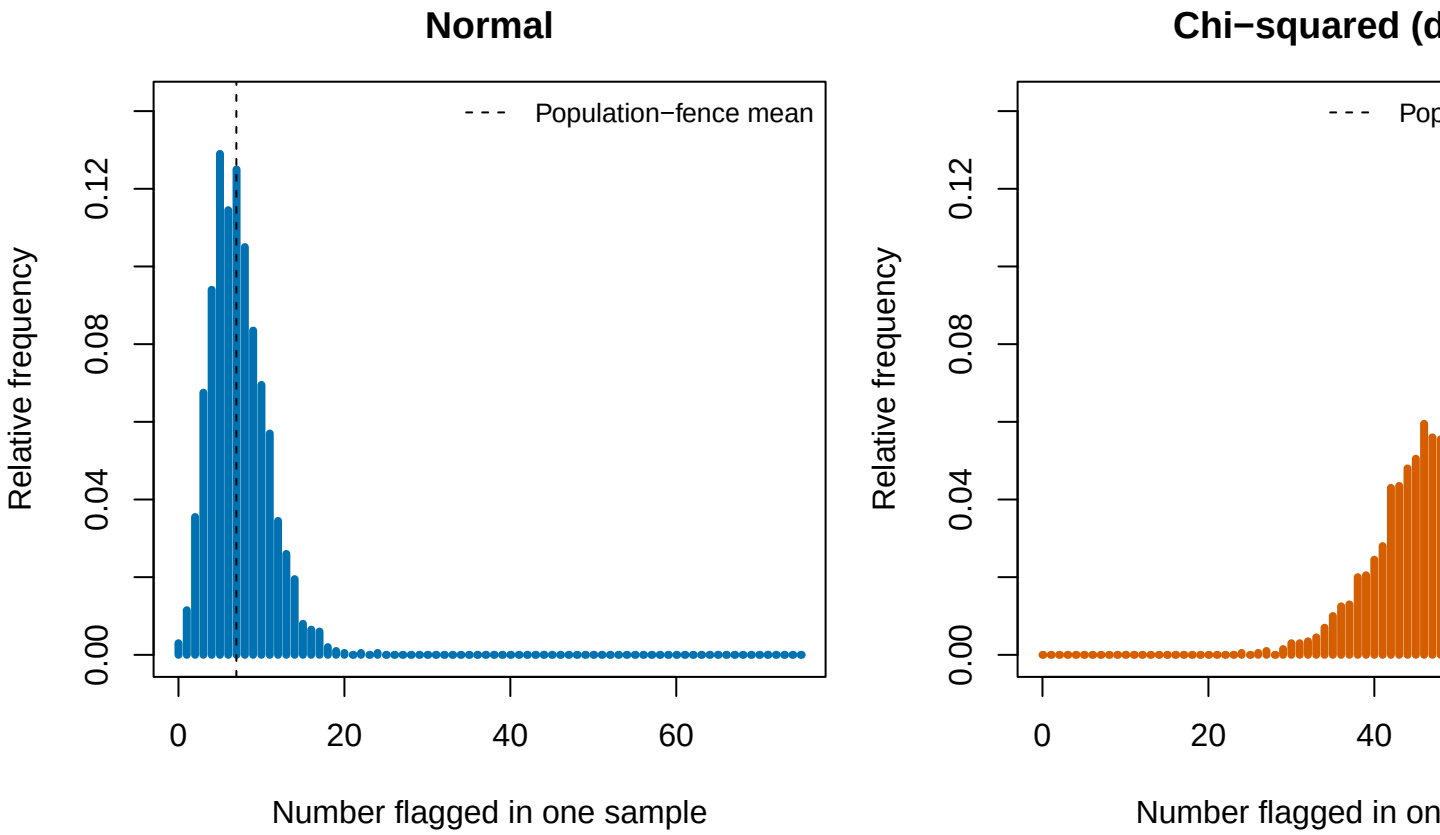



Figure 3: Repeated samples of size 1000. Both models generate flags without any deliberately inserted anomalies.

## How does sample size affect the points flagged by a boxplot?

The simulation distinguishes three questions:

- How variable is the percentage flagged across repeated samples?
- How do the number and percentage flagged change with sample size?
- How often does a sample contain at least one flagged observation?

```
model_order <- names(generators)
sample_sizes <- sort(unique(sim$n))

model_colours <- setNames(
  c("#0072B2", "#D55E00"),
  model_order
)

sim_plot <- sim
sim_plot$Model <- factor(sim_plot$Model, levels = model_order)
sim_plot$Sample_size <- factor(sim_plot$n, levels = sample_sizes)
sim_plot$Percent <- 100 * sim_plot$proportion

summary_plot <- sim_summary
summary_plot$Model <- factor(summary_plot$Model, levels = model_order)
summary_plot$Sample_size <- factor(summary_plot$n, levels = sample_sizes)

# Use model labels from bench, so references remain correct if rows move.
reference_plot <- data.frame(
  Model = factor(bench$Model, levels = model_order),
  Percent = 100 * bench$p_flag
)

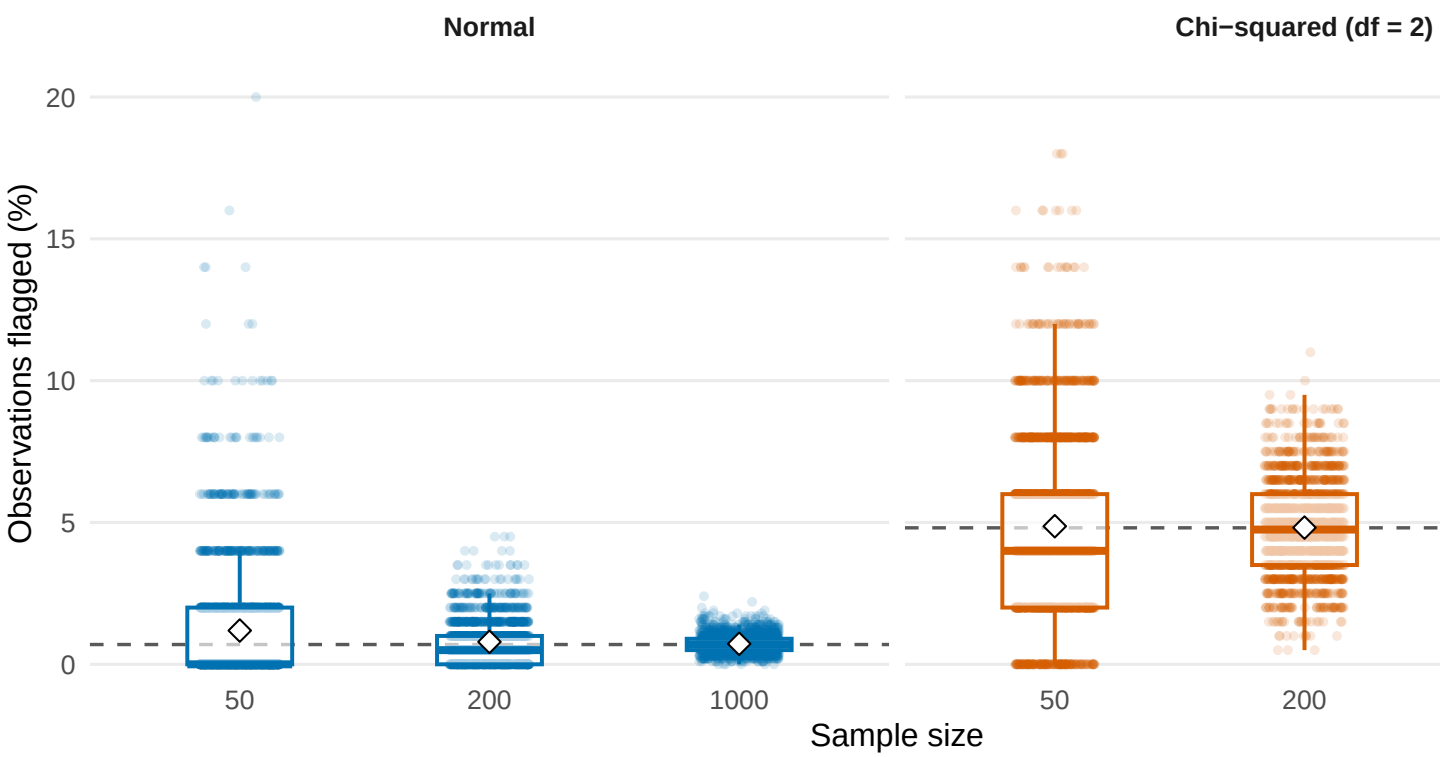
flag_theme <- theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    panel.grid.major.x = element_blank(),
    strip.text = element_text(face = "bold"),
    legend.position = "none",
    plot.title = element_text(face = "bold"),
    plot.title.position = "plot",
    plot.caption = element_text(hjust = 0, colour = "grey40")
  )
```

### The percentage flagged across repeated samples

Each coloured point represents **one simulated sample**. Its vertical position gives the percentage of observations flagged in that sample. Horizontal jitter separates overlapping points without changing their percentages.

The boxplots summarise these simulation results, and the white diamonds mark their means. The dashed lines show the probabilities obtained using population quartiles: approximately **0.70%** for the normal distribution and **4.81%** for the chi-squared distribution with two degrees of freedom.

```r
ggplot(
  sim_plot,
  aes(x = Sample_size, y = Percent, colour = Model)
) +
  geom_hline(
    data = reference_plot,
    aes(yintercept = Percent),
    linetype = "dashed",
    colour = "grey35",
    linewidth = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.16,
      height = 0,
      seed = 314159
    ),
    alpha = 0.15,
    size = 1
  ) +
  geom_boxplot(
    width = 0.42,
    outlier.shape = NA,
    fill = "white",
    alpha = 0.65,
    linewidth = 0.7
  ) +
  stat_summary(
    fun = mean,
    geom = "point",
    shape = 23,
    size = 2.8,
    fill = "white",
    colour = "black"
  ) +
  facet_wrap(~ Model, nrow = 1) +
  scale_colour_manual(values = model_colours) +
  scale_y_continuous(
    limits = c(0, NA),
    expand = expansion(mult = c(0.02, 0.08))
  ) +
  labs(
    title = "Larger samples give more stable percentages",
    subtitle = "Each point is one simulated sample; diamonds indicate means",
    x = "Sample size",
    y = "Observations flagged (%)",
    caption = paste(
      "Dashed lines: population-quartile benchmarks.",
      "Both panels use the same vertical scale."
    )
  ) +
  flag_theme
```




Dashed lines: population−quartile benchmarks. Both panels use the same vertical scale.

**Interpretation.** As sample size increases, the percentages become more concentrated around their distribution-specific benchmarks. They do not approach zero: even an uncontaminated sample can contain observations beyond the boxplot fences.

For smaller samples, the percentages occur in visible steps. With $n = 50$, one flagged observation corresponds to 2%; with $n = 1000$, it corresponds to 0.1%.

The population benchmarks are limiting references, not exact finite-sample expectations: each simulated sample uses its own estimated quartiles and fences.

> Here, each boxplot summarises percentages from repeated simulations. It does not display the original observations within a single sample.

### More flagged observations does not imply a higher percentage

The following display compares **counts** and **percentages**. Points indicate simulation means; vertical intervals extend from the 5th to the 95th percentile of the simulation results.

These intervals describe variation between samples. They are **not confidence intervals for the means**.

```
comparison_plot <- rbind(
  transform(
    summary_plot,
    Metric = "Number flagged",
```

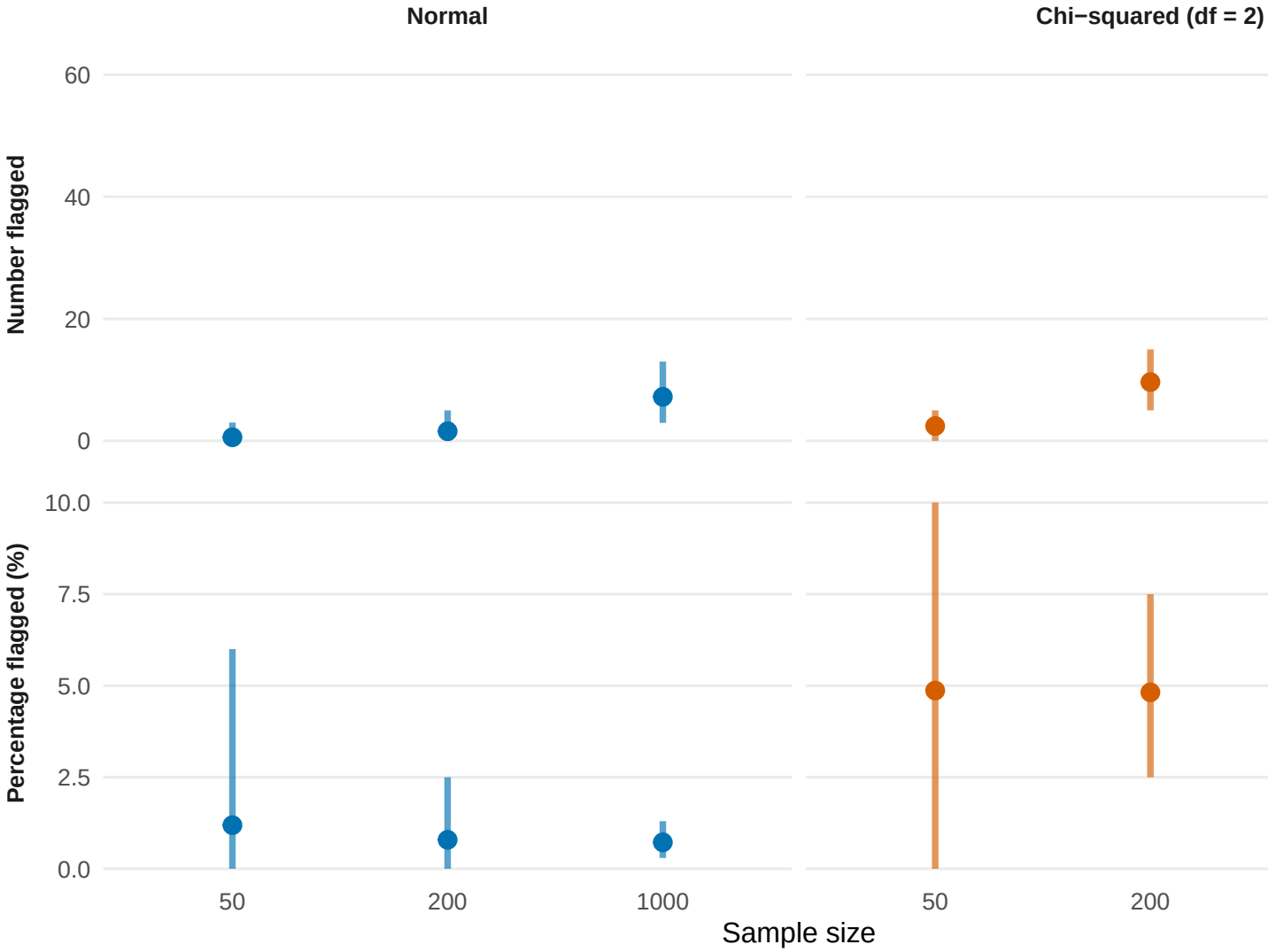
```
    Mean = Mean_count,
    Low = Count_q05,
    High = Count_q95
  ),
  transform(
    summary_plot,
    Metric = "Percentage flagged (%)",
    Mean = Mean_percent,
    Low = 100 * Count_q05 / n,
    High = 100 * Count_q95 / n
  )
)

comparison_plot$Metric <- factor(
  comparison_plot$Metric,
  levels = c("Number flagged", "Percentage flagged (%)")
)

ggplot(
  comparison_plot,
  aes(x = Sample_size, y = Mean, colour = Model)
) +
  geom_linerange(
    aes(ymin = Low, ymax = High),
    linewidth = 1.3,
    alpha = 0.65
  ) +
  geom_point(size = 3.2) +
  facet_grid(
    Metric ~ Model,
    scales = "free_y",
    switch = "y"
  ) +
  scale_colour_manual(values = model_colours) +
  scale_y_continuous(
    limits = c(0, NA),
    expand = expansion(mult = c(0.03, 0.10))
  ) +
  labs(
    title = "Counts increase while percentages stabilise",
    subtitle = "Points: means; intervals: 5th-95th percentiles across samples",
    x = "Sample size",
    y = NULL,
    caption = paste(
      "The two rows use different vertical scales.",
      "Within each row, both models share the same scale."
    )
  ) +
  flag_theme +
  theme(strip.placement = "outside")
```

## Counts increase while percentages stabilise

Points: means; intervals: 5th–95th percentiles across samples




The two rows use different vertical scales. Within each row, both models share the same scale.

**Interpretation.** Larger samples generally contain more flagged observations, but this does not imply that a larger fraction of the data is unusual.

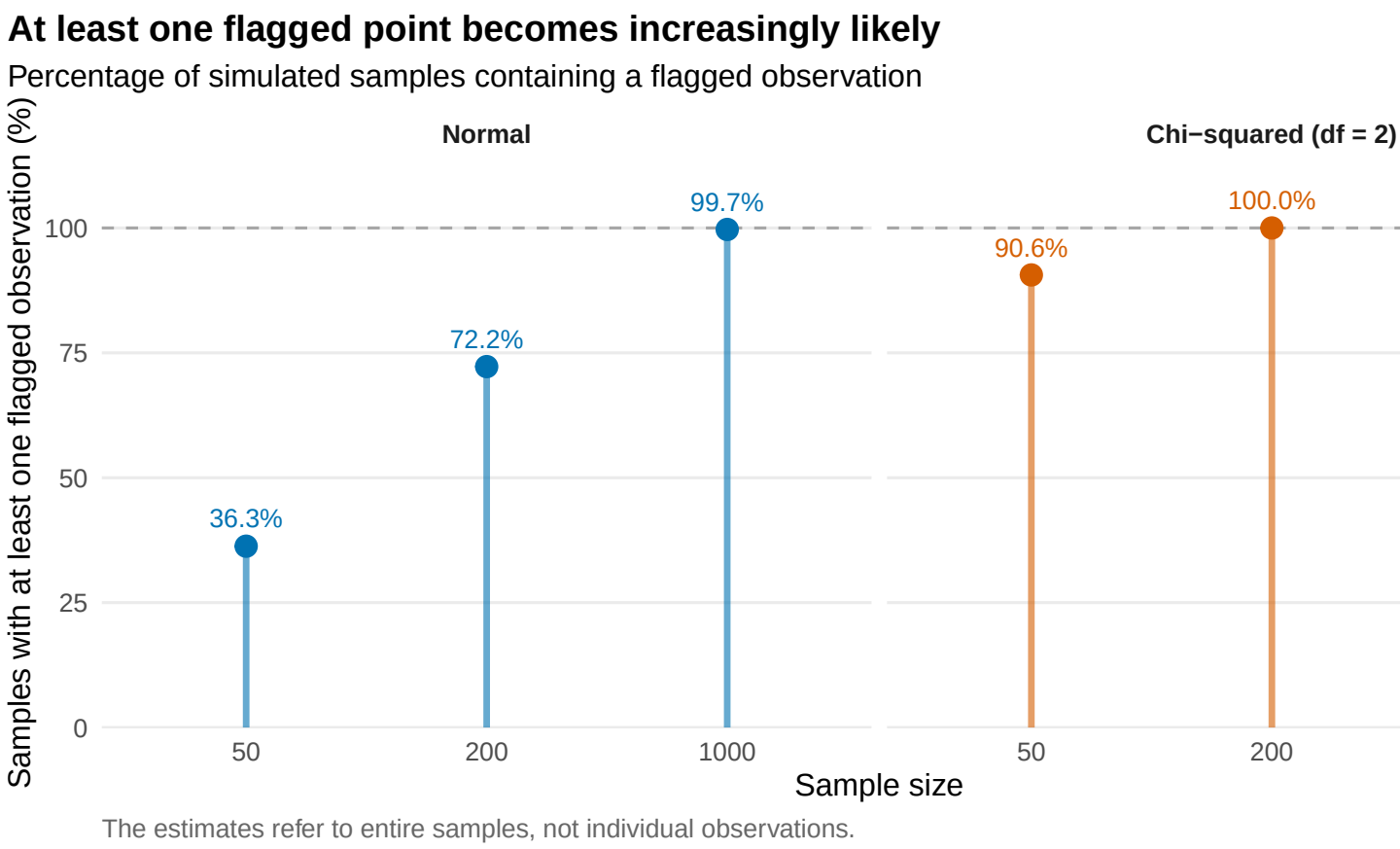
Compare the intervals in the two rows. The absolute variability of the counts can increase while the variability of the percentages decreases. A large sample can therefore contain many flagged observations and still have a very predictable percentage flagged.

Increasing the number of simulation repetitions, $B$, estimates these intervals more precisely; it does not make the underlying variation between samples disappear.

### How often is at least one observation flagged?

For each sample size and distribution, we estimate the probability of observing at least one flagged point by the proportion of simulated samples in which this occurs.

```r
ggplot(
  summary_plot,
  aes(
    x = Sample_size,
    y = 100 * Pr_any,
    colour = Model
  )
) +
  geom_hline(
    yintercept = 100,
    linetype = "dashed",
    colour = "grey65"
  ) +
  geom_segment(
    aes(
      xend = Sample_size,
      y = 0,
      yend = 100 * Pr_any
    ),
    linewidth = 1.2,
    alpha = 0.6
  ) +
  geom_point(size = 3.5) +
  geom_text(
    aes(label = sprintf("%.1f%%", 100 * Pr_any)),
    vjust = -1,
    size = 3.5,
    show.legend = FALSE
  ) +
  facet_wrap(~ Model, nrow = 1) +
  scale_colour_manual(values = model_colours) +
  scale_y_continuous(
    limits = c(0, 100),
    breaks = seq(0, 100, 25),
    expand = expansion(mult = c(0, 0.14))
  ) +
  labs(
    title = "At least one flagged point becomes increasingly likely",
    subtitle = "Percentage of simulated samples containing a flagged observation",
    x = "Sample size",
    y = "Samples with at least one flagged observation (%)",
    caption = "The estimates refer to entire samples, not individual observations."
  ) +
  flag_theme
```




**Interpretation.** A small percentage of observations flagged can coexist with a high probability of seeing at least one flagged observation in a sample.

For both distributions considered here, this probability approaches one as sample size increases. The presence of points beyond the whiskers, by itself, is therefore not evidence of data errors or contamination.

**Check your understanding:** how can approximately 0.7% of individual observations be flagged under normality, while almost every sufficiently large normal sample contains at least one flagged observation?

### How to interpret the simulation

- **The model matters.** The right-skewed chi-squared model produces a larger flagged fraction than the normal model with the same boxplot rule.
- **The sample size matters.** Larger samples usually contain more flagged observations, even when the fraction is approximately stable.
- **Sample fences fluctuate.** At small $n$, their variability can make the average flagged fraction differ noticeably from its population benchmark.
- **Replication matters.** The result from one sample is not a general property of its generating model. Compare averages and their Monte Carlo uncertainty.

#### Are the chi-squared flags really outliers?

They are outliers *according to the boxplot rule*. In this simulation, they are valid draws from the very same distribution as all the other observations. There is no separate anomalous population and no recording error. Some tail values are expected under the model; the IQR rule is not calibrated to give the same tail probability across distributions.

This does not imply that every positive value is equally plausible under a chi-squared model. An extremely large value could still be surprising under that model. Evaluate its tail probability and the number of opportunities for an extreme value, rather than equating every boxplot flag with an error. Nor does a reasonable total number of flags certify that every measurement is correct.

## Whenever feasible, show the points as well

### Boxplot plus observations

The boxplot supplies a compact summary; the points reveal how observations populate that summary. Their combination is especially useful for small or moderate datasets.

```
set.seed(271828)
local({
  par(mfrow = c(1, 2), mar = c(3, 4, 3, 1))
  box_points(samples[[1]], main = "Normal: summary + observations", col = blue)
  box_points(samples[[2]], main = "Chi-squared: summary + observations", col = orange)
})
```

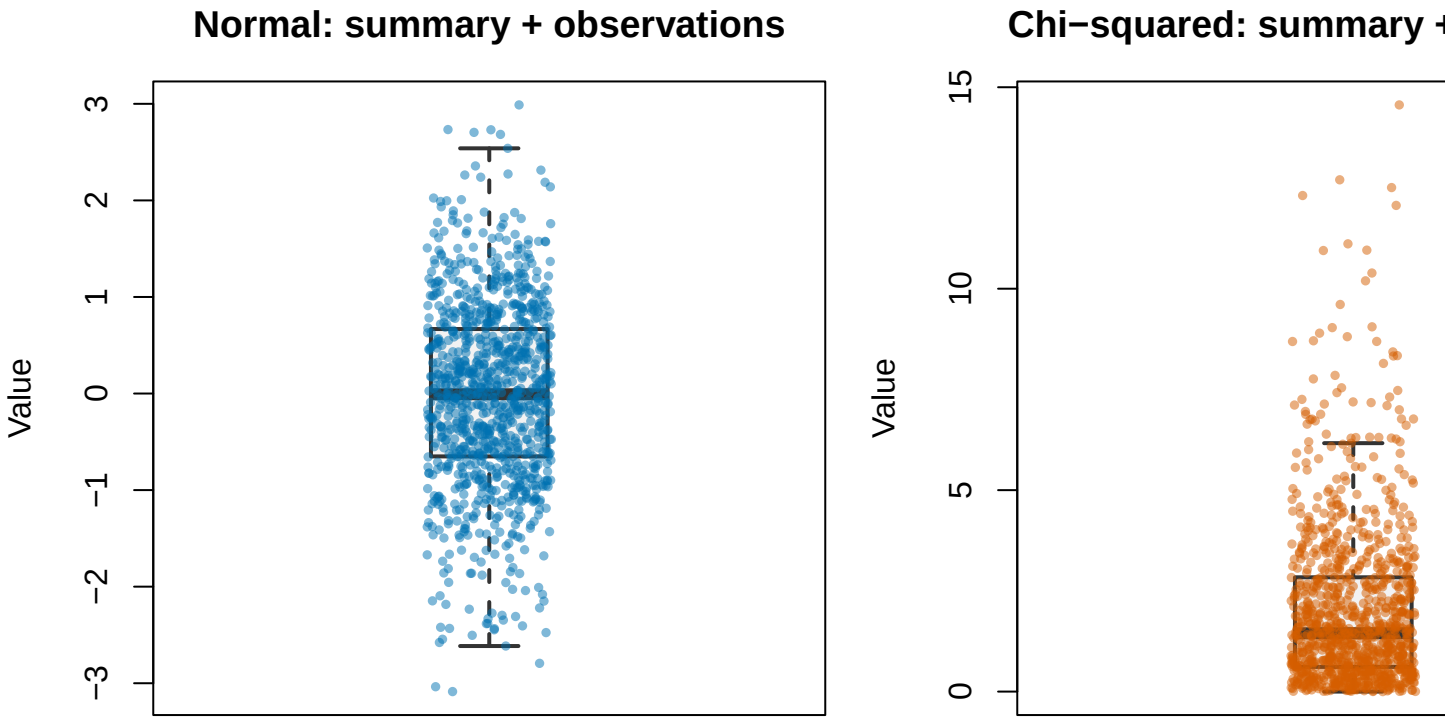



Figure 4: All observations are shown once. Horizontal jitter reduces overlap without changing measured values.

The helper `box_points()` uses `outline = FALSE` to suppress the boxplot's separate symbols, then draws **every** observation. This avoids plotting the flagged observations twice. It explicitly sets `ylim = range(z)` so that the full dataset remains visible.

Jitter is applied only along the meaningless horizontal position within a category. **Do not jitter the measured vertical values** merely to separate points. Use small symbols and transparency when observations overlap. For very large samples, an ECDF, histogram, density plot or another distributional display may be clearer; disclose any subsampling of a point layer.

## Same boxplot, different internal structure

The next example is constructed so that two datasets have identical boxplot summaries. One fills the range quite evenly; the other contains conspicuous gaps and repeated values. Can you recover that difference from a box alone?

```
even <- seq(0, 20, length.out = 41)
gapped <- c(rep(0, 10), 5, rep(5, 9), 10, rep(15, 9), 15, rep(20, 10))
stopifnot(length(even) == length(gapped),
          isTRUE(all.equal(boxplot.stats(even)$stats,
                           boxplot.stats(gapped)$stats)))
knitr::kable(data.frame(
  Element = c("Lower whisker", "Lower edge", "Median", "Upper edge", "Upper whisker"),
  Even = boxplot.stats(even)$stats,
  Gapped = boxplot.stats(gapped)$stats
), row.names = FALSE)
```

| Element | Even | Gapped |
|---|---|---|
| Lower whisker | 0 | 0 |
| Lower edge | 5 | 5 |
| Median | 10 | 10 |
| Upper edge | 15 | 15 |
| Upper whisker | 20 | 20 |

```
set.seed(161803)
local({
  par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
  groups <- list(Even = even, Gapped = gapped)
  boxplot(groups, col = "grey92", ylab = "Value", main = "Boxplots alone")
  boxplot(groups, outline = FALSE, col = "grey95", ylim = range(even, gapped),
          ylab = "Value", main = "Add the observations")
  for (j in seq_along(groups)) {
    points(jitter(rep(j, length(groups[[j]])), amount = 0.1), groups[[j]],
           pch = 16, cex = 0.7, col = adjustcolor(blue, alpha.f = 0.6))
  }
})
```

This deliberately simple construction isolates the information lost by a five-number summary. For further examples, see Wilke's discussion of boxplots, violins and individual points.

### The same example in ggplot2

This is the ggplot2 equivalent of the preceding point-overlay plot. Its default quartile convention can differ slightly from base R's hinges, especially in small samples. For these two datasets the summaries agree.

```
plot_data <- data.frame(
  groups = rep(c("Even","Gapped"),c(length(even),length(gapped))),
  Y = c(even,gapped)
)
ggplot(plot_data, aes(x = groups, y = Y)) +
  geom_boxplot(outlier.shape = NA, width = 0.4, fill = "grey95") +
```

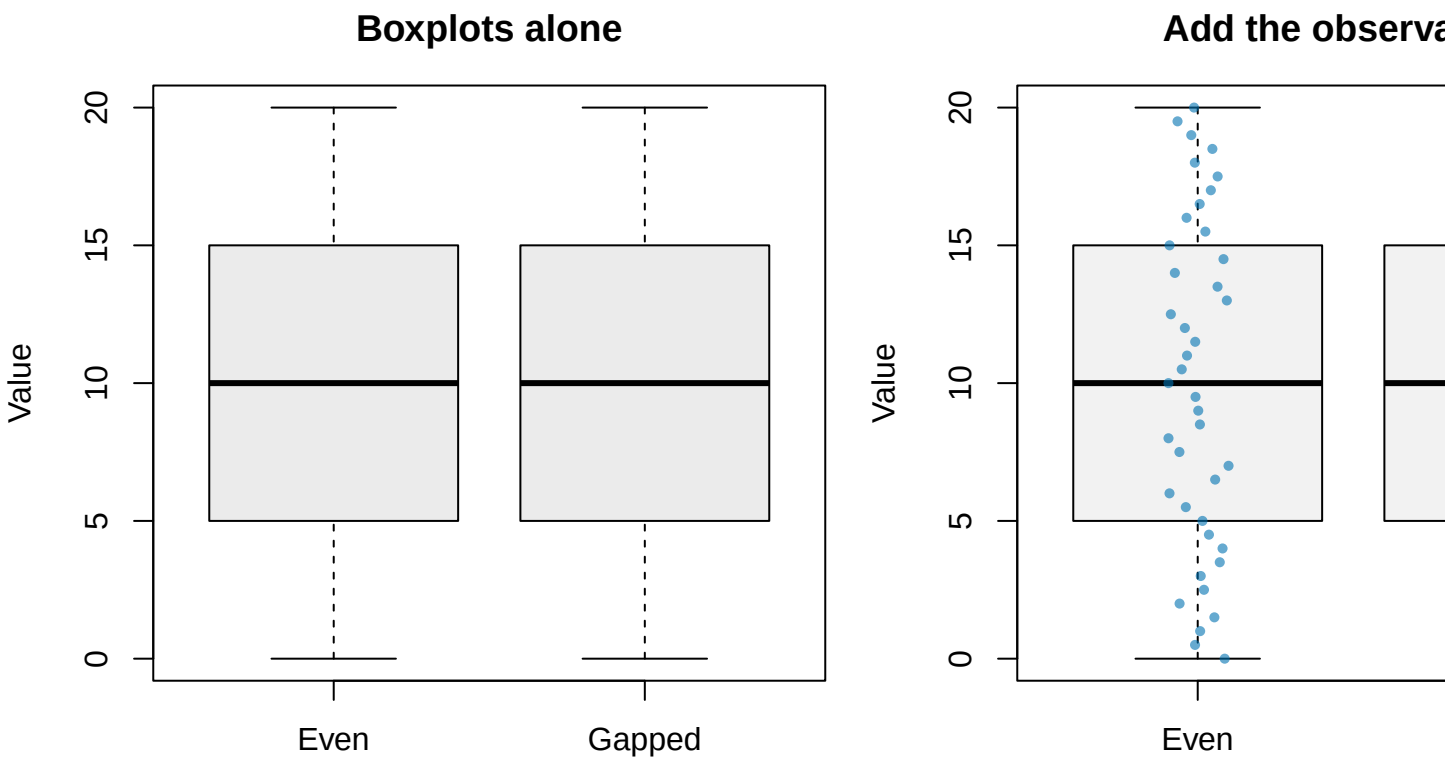
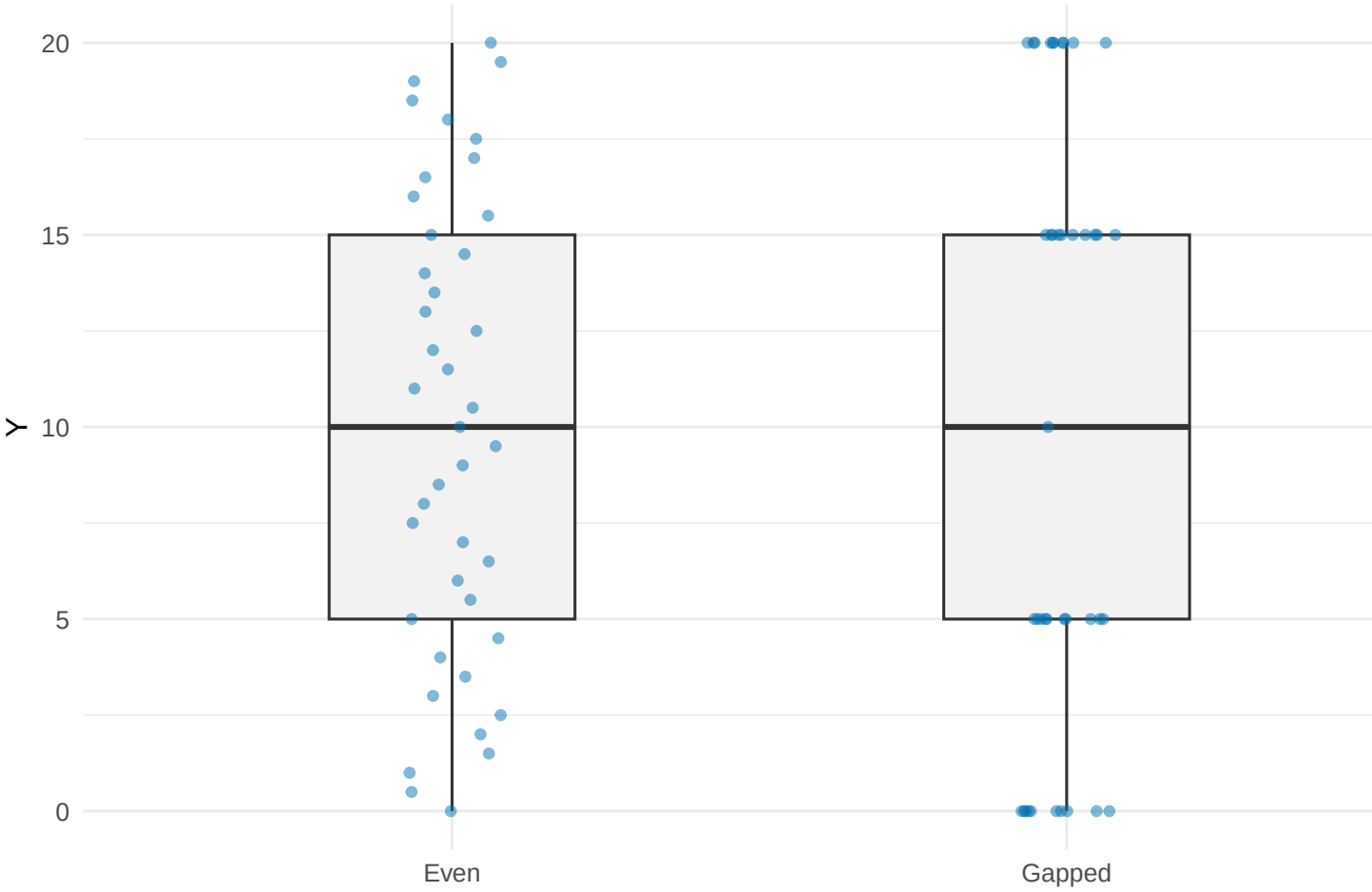



Figure 5: Identical boxplots conceal different arrangements of the observations. Each dataset has n = 41.

```
geom_point(position = position_jitter(width = 0.08, height = 0, seed = 271828),
           alpha = 0.5, size = 1.5, colour = "#0072B2") +
labs(x = NULL, y = "Y") +
theme_minimal(base_size = 12)
```




Here `outlier.shape = NA` hides only the boxplot's isolated symbols; the point layer still shows all observations. See the geom_boxplot documentation.

## Take-home messages

**A boxplot is a summary of a distribution. A flagged observation is a reason to investigate, not an automatic reason to remove it.**

1. **Read the elements correctly.** The box describes roughly the middle 50% of the observations, and its internal line is the median. Fences are calculated thresholds; whiskers end at observed values within those thresholds. Neither the box nor the whiskers is a confidence interval.
2. **Interpret flags relative to the generating distribution.** With population quartiles, the 1.5-IQR rule flags about **0.70%** of a normal population but **4.81%** of a chi-squared population with two degrees of freedom. These observations can be valid tail values. The rule does not define a universal significance level or identify measurement errors.
3. **Separate counts, percentages and the probability of at least one flag.** For the two models studied here, larger samples generally contain more flagged observations and are more likely to contain at least one. The percentage flagged becomes more stable around a positive, distribution-specific value; it does not tend to zero.

4. **Distinguish population benchmarks from sample boxplots.** Actual boxplots estimate their fences from the same observations they classify. The fences fluctuate, especially in small samples, and the flags are dependent. The binomial count calculation is exact for independent observations with fixed population fences, not generally for estimated sample fences.

5. **Distinguish sample variability from simulation precision.** The standard deviation and 5th–95th percentile intervals describe variation across samples. The Monte Carlo standard error describes uncertainty in the simulated mean. Increasing sample size $n$ changes the sampling distribution; increasing repetitions $B$ estimates it more precisely.

6. **Show what the summary hides.** Identical boxplots can conceal different gaps, clusters and repeated values. When feasible, add all observations, using jitter only along the categorical axis, and report sample sizes. Interpret unusual values using the measurement context and the model before deciding how to handle them.

## Sources and further reading

- R documentation: Box Plot Statistics. Authoritative details of base R hinges and whiskers.
- Wilke, C. O. *Fundamentals of Data Visualization*: Visualizing many distributions at once. Examples comparing distribution summaries and individual observations.
- ggplot2 documentation: A box and whiskers plot. Quartile conventions and point-overlay options.

The numerical population calculations, simulations and teaching datasets in this unit are developed explicitly in the code above.

---

**Note on AI assistance**

These teaching materials were prepared with assistance from ChatGPT (OpenAI) in drafting explanations, developing examples, and generating R code. The author retains responsibility for the final content.